

팀명: 노크온
팀장: 전민수
팀원 : 홍진원, 김희정

버튼 시퀀스·노크 리듬 기반 저비용 이중 인증(2FA) 잠금 시스템

MCU 레지스터 직접 제어와 행동 기반 인증으로 구현한 실용적 보안 설계

기존 저비용 잠금장치의 한계

1

관찰 노출



PIN 입력을 옆에서 지켜보는 것만으로 해킹
물리적 침입, 해킹 불필요

2

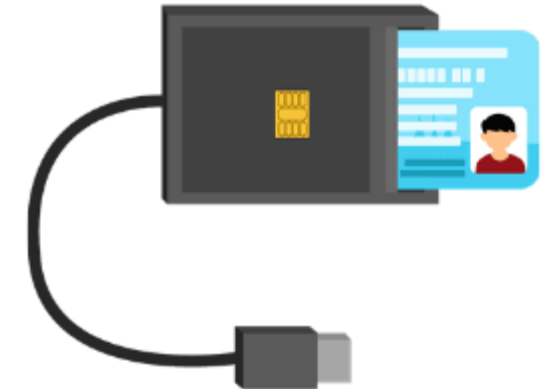
물리적 흔적 노출



자주 누르는 버튼만 닳거나 지문 얼룩이 남아,
후보 자리 유추 가능

3

매체 분실·복제 위험



저가 125kHz 카드류는 복제기로 복제 가능
카드 분실시 시스템 전체 무력화

4

비가역성



인증수단인 생체 정보가 유출 시 고유 정보인
지문 등 변경 불가로 인한 취약성

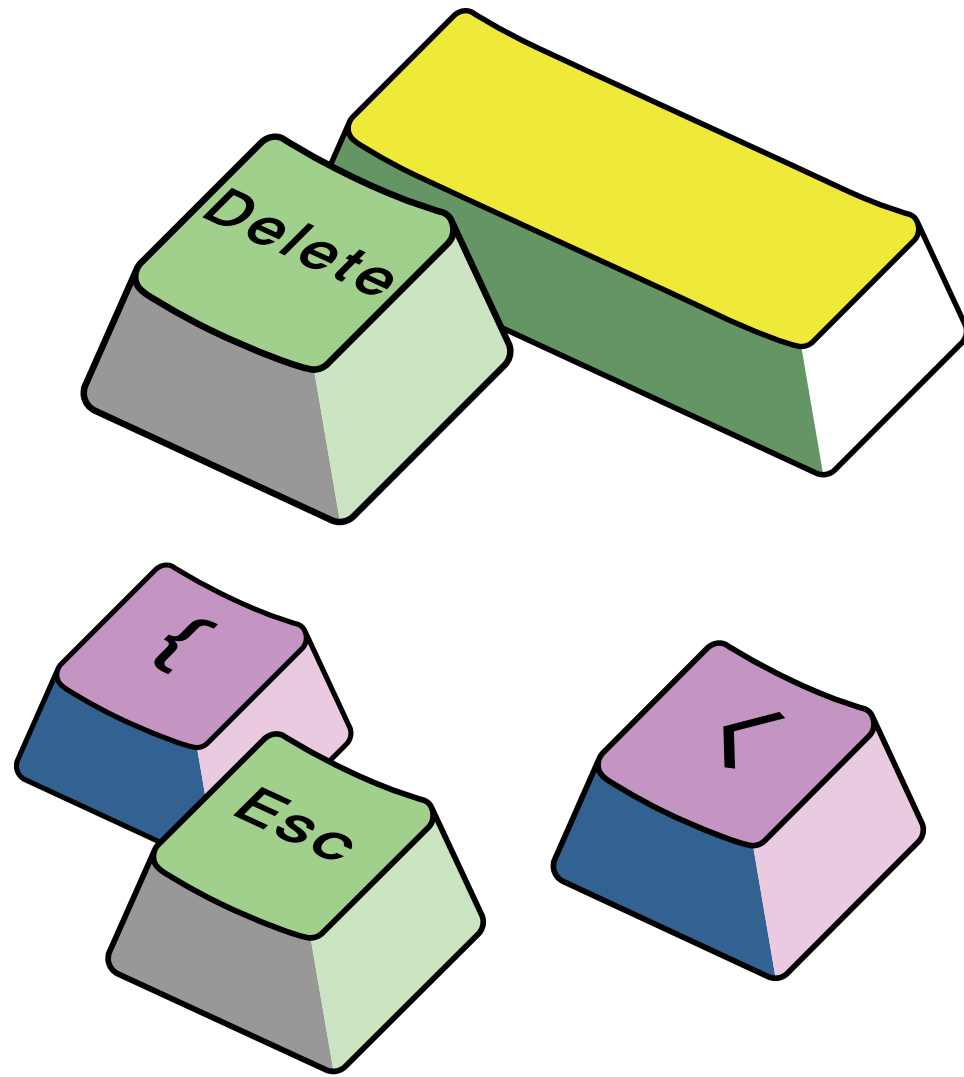
5

단일 인증 계층 구조



인증수단이 하나라면 검증절차 없이 보안 무력화

Keystroke Dynamics에서 착안한 행동 기반 인증



Keystroke Dynamics란?

타이핑 습관과 입력 간격으로 사용자를 구분하는 행동 인증 기법

순서와 리듬의 이중 검증

서로 다른 주변 장치에서 독립적으로 얻어지는 두 계층(버튼, 리듬)을 통해
동시 유출 가능성 제거

디지털 GPIO 기반 4개 버튼 순서 입력
4자리 조합으로 256가지 패턴 생성 가능

+

아날로그 비교기 및 ADC를 활용한 리듬 입력
두드리는 리듬을 측정하여 입력 방식 확장

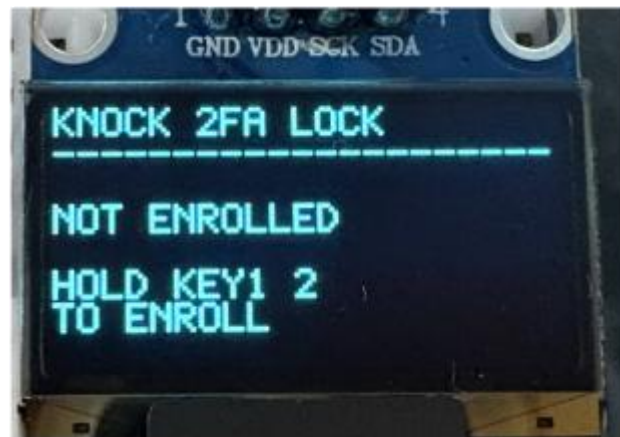
저비용 행동 패턴 인증

약 500원 비용의 압전 소자 하나를 통해 행동 패턴을 파악하고
저비용 8비트 MCU 2FA 효과 구현

동작원리

1

초기화 단계



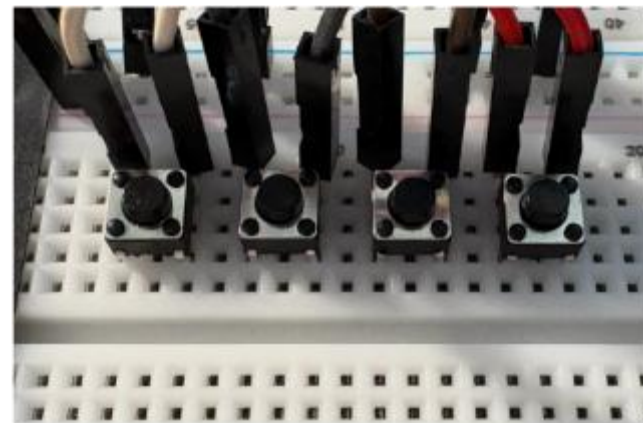
판정모드/ 등록모드

등록 이전) Press KEY1 (2s)
버튼 순서, 리듬 5회 반복 입력

등록 이후) Press random key
대기 모드의 MCU 활성화(10s)

2

버튼 입력 단계



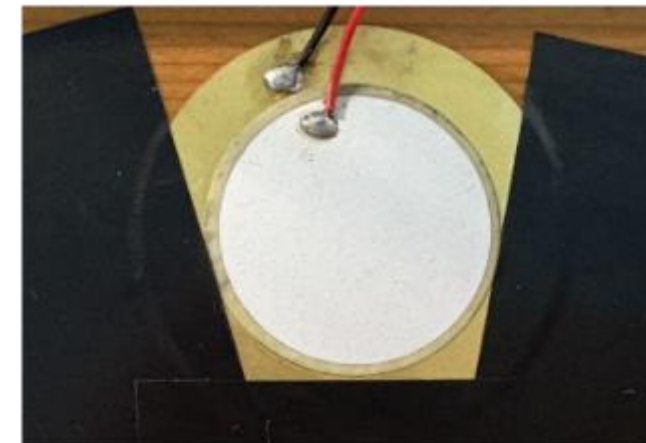
버튼 4개 순서 차례대로 입력

정답 입력 시) 다음 단계로 이동
오답 입력 시) 인증 거부

EEPROM 실패 카운터

3

리듬 측정 단계

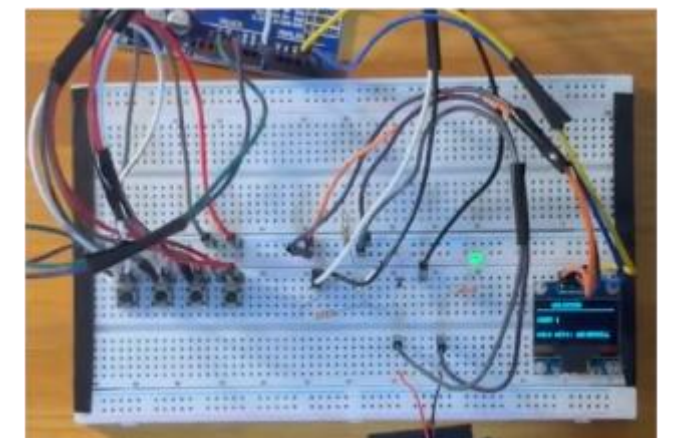


4번의 노크리듬 입력

리듬에 의한 피에조 센서의 전압 변화
→ MCU 내부의 아날로그
비교기 감지, ALD 진폭 측정
→ Timer1 입력 캡처(ACIC=1),
타격 시점 기록(500ns)

4

판정 단계



독립 검증

1단계 & 2단계 모두 통과 시
액추에이터 핀에 신호 전달
→ 잠금 해제(3s)
실패 카운터 초기화

상세 동작원리 - 2단계 & 3단계

2

버튼 입력 단계

소프트웨어 디바운스

손가락이 떨어져서 두 번 눌리는 걸 막기
위해 20ms 간격으로 신호를 감지

3

리듬 측정 단계

불응기

타격에 의한 진동 오감지 방지
첫 타격 직후 모든 신호 무시(250ms)

템포의 정규화

상황에 따라 빠르기가 달라질 수 있기 때
문에 절대적인 시간이 아닌 간격의 비율
을 구하여 판단



knock.c — TIMER1_CAPT_vect (요약)

```
ISR(TIMER1_CAPT_vect) {
    uint16_t icr = ICR1; /* 하드웨어가 이미 래치해 둔 시각 */
    uint16_t ovf = g_ovf;

    /* 오버플로 경계 보정 - 캡처(벡터10)가 오버플로(벡터13)보다
       우선순위가 높아, TOV1 이 선 채로 여기 들어올 수 있다.
       보정하지 않으면 32.768 ms 단위 이상값이 간헐적으로 섞인다. */
    if ((TIFR1 & (1 << TOV1)) && (icr < 0x8000)) ovf++;
    now = ((uint32_t)ovf << 16) | icr;

    /* 불응기 안이면 같은 노크의 잔향이므로 버린다 */
    if ((uint32_t)(now - g_last_ts) < BLANK_TICKS) return;
    g_last_ts = now;
    g_pend_ts = now; g_pend = 1; /* 진폭 검사는 메인 루프에서 */
}
```

knockfp.c — knockfp_make() 핵심부

```
/* 곱셈이 32비트에 들어갈 때까지 전 성분을 함께 우측 시프트한다.
   동일 배율이므로 성분 간 비율(= 리듬의 형태)은 보존된다.
   MAX_TOTAL = 5000 ms = 1e7 틱 -> 시프트가 한 번은 필요할 수 있다.
   시프트 후 total <= 4e6 이므로 g[k]*1000 <= 4.0e9 < 4.294e9. */
while (total > KNOCKFP_SHIFT_LIMIT) {
    total = 0;
    for (k = 0; k < CLASSIFY_CHANNELS; k++) {
        g[k] >>= 1;
        total += g[k];
    }
}

for (k = 0; k < CLASSIFY_CHANNELS; k++) {
    fp->n[k] = (uint16_t)((g[k] * CLASSIFY_NORM_SUM) / total);
} /* NORM_SUM = 1000 */
fp->status = CLASSIFY_OK;
```

상세 동작원리 - 4단계

1

버튼 입력 단계 불일치

pin_user = -1이므로 다음 단계로 넘어갈 수 없다

2

리듬 측정 단계 불일치

버튼 입력이 옳더라도

cls ≠ pin_user이므로 해제되지 않는다

3

반복된 무차별 시도

5회 실패시 30초 하드락아웃으로 보안

실패 시 카운터를 RAM이 아니라 EEPROM에 먼저

기록하므로, 전원을 껐다 켜도 락아웃이 풀리지 않는다.

main.c — run_auth() 발췌

```
/* [보안] 평가하기 전에 실패 카운터를 먼저 올린다.  
평가 후에 올리면, 결과가 확정되기 직전에 전원을 끄어  
카운터를 회피하고 무제한으로 시도할 수 있다. */  
authstore_fail_set(authstore_fail_get() + 1);  
  
pin_user = pin_lookup(g_pin_buf);          /* 1단계 결과 */  
thr = (pin_user >= 0) ? g_db.threshold_d2[pin_user] : 0;  
  
if (take_rhythm(&kr, &fp)) {               /* 2단계 수집 */  
    cr = classify_match(&fp, g_db.centroid, thr);  
  
    /* 두 인증 계층이 같은 사용자를 가리켜야 한다.  
    순서만 맞고 리듬이 다르면 cr.cls != pin_user 로 거부된다.  
    cr.cls 는 임계값 초과 시 -1 이므로, 아래 한 줄이  
    "둘 다 성립하고 둘이 같다" 만 통과시킨다. */  
    if (pin_user >= 0 && cr.cls == pin_user) {  
        granted = 1; *user = pin_user;  
    }  
}  
  
if (granted) authstore_fail_set(0); /* 성공 시에만 복구 */
```

4

판정 단계

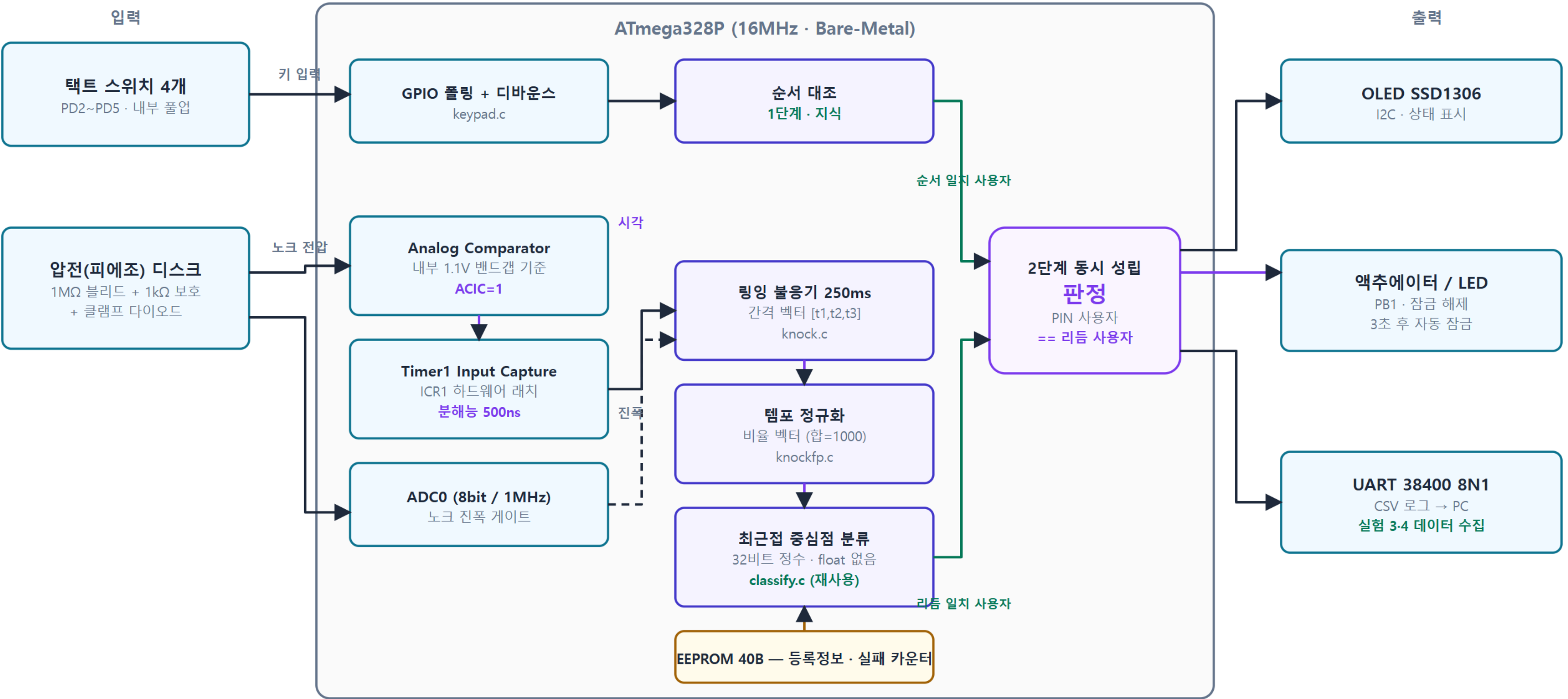
하드락아웃 대기

5번 연속 실패시

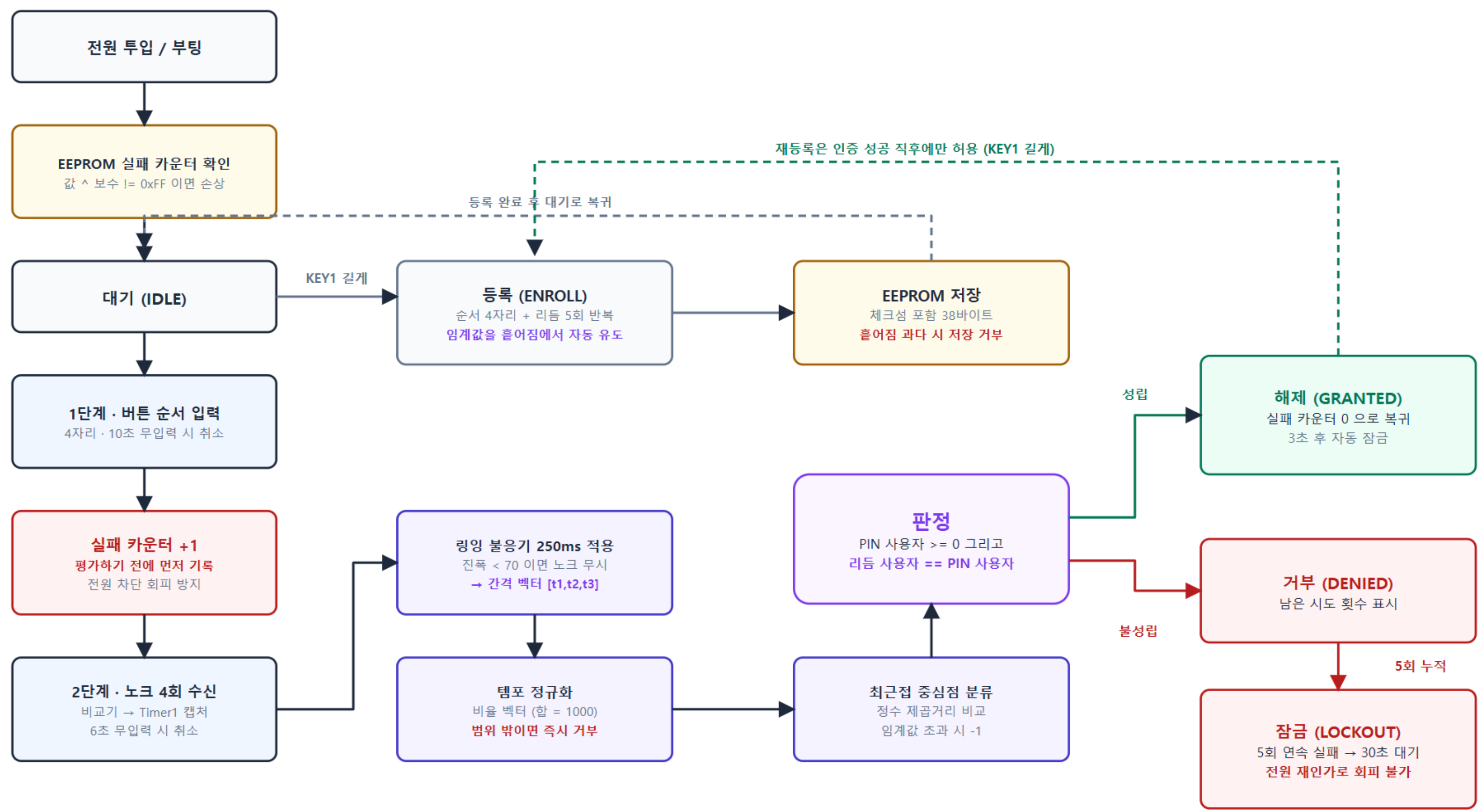
전원 연결 여부와 관계 없이

하드락아웃 대기 상태 진입(30s)

시스템 블록도



프로그램 흐름



구현 전략

: 소프트웨어 + 하드웨어



불필요한 외부 부품 없이
필요한 구현 시스템을 완벽하게
갖춘 최적화 하드웨어·소프트웨어
가성비 2FA 보안 시스템

하드웨어 구현

MCU

ATmega328P
= ADC + EEPROM + 비교기 등

입력 및 센서부

입력(버튼 4개) + 센서(압전 디스크)
+ 블리드 저항 + 보호 다이오드

출력 및 확장

OLED 디스플레이
액추에이터/LED 라인

소프트웨어 구현

Bare-level 직접 제어

레지스터를 직접 제어를 통한
가볍고 빠른 시스템

모듈 재사용과 정수 연산

검증한 모듈의 효율적 반복 사용
메모리 소모가 큰 계산 배제

이중 보안 및 방어 로직

타이머 입력 캡처와 EEPROM 통한
무차별 대입 공격 차단

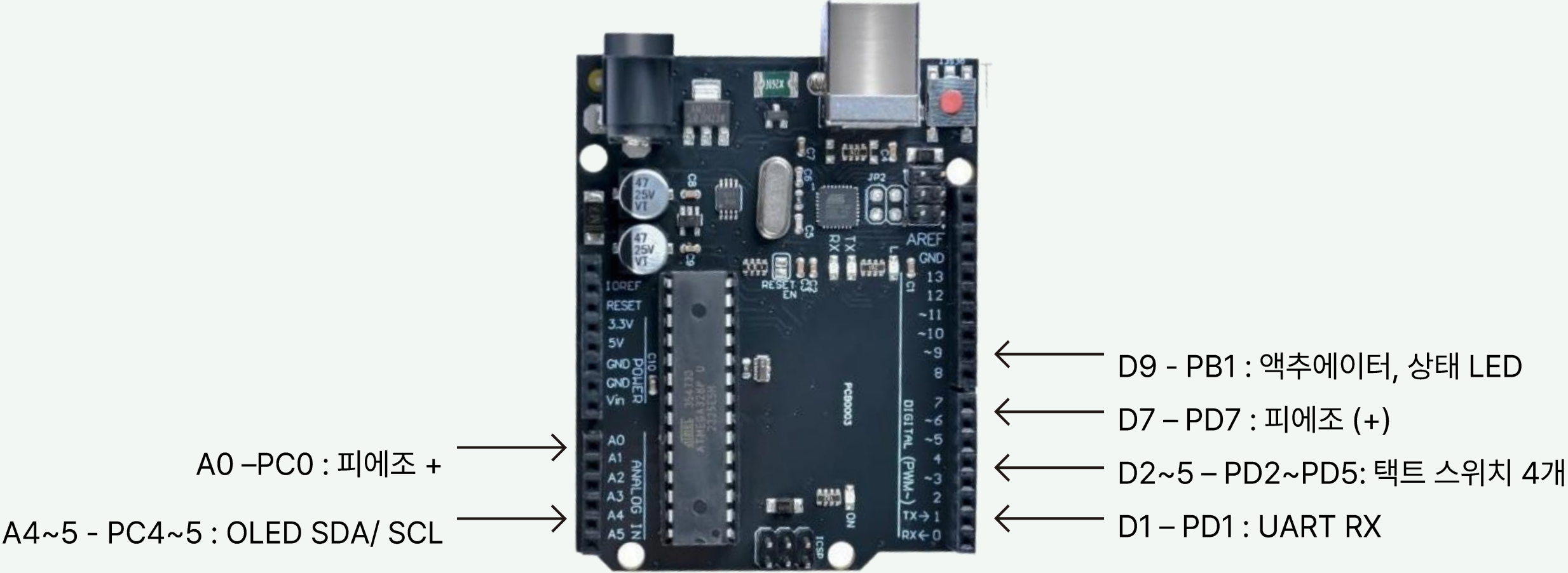
하드웨어 구현

MCU
ATmega328P
= ADC + EEPROM + 비교기 등

입력 및 센서부
입력(버튼 4개) + 센서(압전 디스크)
+ 블리드 저항 + 보호 다이오드

출력 및 확장
OLED 디스플레이
액추에이터/LED 라인

ATmega328P - 16MHz. Bare-Metal



ATmega328P의 장점

- Bare-Metal 레지스터 직접 제어: 외부 라이브러리 전면 배제 + 하드웨어 레지스터 직접 제어로 실행 속도 극대화
- 하드웨어 캡처 경유 (ACIC): 아날로그 비교기 출력을 Timer1에 직결하여 소프트웨어 지터 없이 500ns 분해능 확보
- 추가 외부 IC 제로 (0개): ADC·EEPROM·TWI·USART 내부 내장으로 진폭 측정, 로그, 비휘발성 카운터 단일 칩 처리
- SRAM 제약 기반 정수 연산: 2KB RAM 한계를 극복하기 위해 부동소수점 대신 32비트 정수 연산 분류기 채택
- 하드웨어 16비트 타이머 확장: 프리스케일러 /8 및 오버플로우 카운터 소프트웨어 처리를 통한 시각 확장
- Idle 슬립 모드 전력 관리: Timer0 1ms 틱 주기로만 깨어나는 대기 전력 최소화 구조

하드웨어 구현

MCU
ATmega328P
= ADC + EEPROM + 비교기 등

입력 및 센서부
입력(버튼 4개) + 센서(압전 디스크)
+ 블리드 저항 + 보호 다이오드

출력 및 확장
OLED 디스플레이
액추에이터/LED 라인

1MΩ 블리드 저항



동작필수저항
잔류 전하 신속 방전을 통한
노드 전위 표류 방지 및
연속 타격 감지 보장

1kΩ 직렬 저항



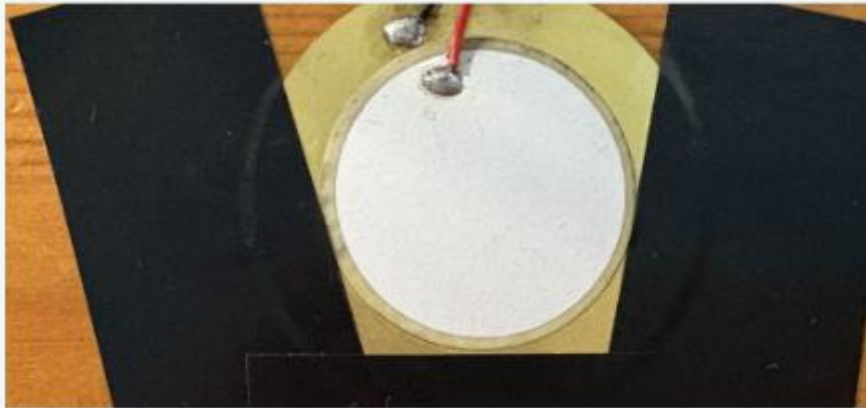
MCU 내부 ESD 클램프 보호
및 과전류 유입 제한

1N4148 클램프 다이오드



음의 스윙 클램핑 및 과전압 보호를
통한 MCU 핀 안전 확보

기계적 고정 및 댐핑재 부착



단단한 면 고정 및 댐핑재 적용으로 공
진에 의한 다중 트리거억제

소프트웨어 구현

Bare-level 직접 제어

레지스터를 직접 제어를 통한
가볍고 빠른 시스템

하드웨어 최적화 및 타이밍 캡처

- knock.c: 아날로그 비교기 인터럽트와 Timer1 입력 캡처(ACIC)를 직결 소프트웨어 지터 없이 500ns 하드웨어 분해능으로 노크 시각과 진폭을 동시 측정
- timebase.c: Timer0 기반의 1ms 시스템 시각 및 Idle 슬립 모드를 지원하여 대기 전력 최소화

모듈 재사용과 정수 연산

검증한 모듈의 효율적 반복 사용
메모리 소모가 큰 계산 배제

기존 함수 재사용을 통한 효율 극대화

- classify.c: 최근접 중심점 정수 분류기
- i2c.c / ssd1306.c: TWI 마스터 및 OLED 텍스트 출력 전담
- uart.c: 38400 8N1 기반 실시간 실험 로그(CSV) 송신

이중 보안 및 방어 로직

타이머 입력 캡처와 EEPROM 통한
무차별 대입 공격 차단

입력 안정성 및 비휘발성 방어

- keypad.c: 4개 텍스트 스위치 20ms 주기 폴링 디바운스 적용, 누르는 시간 감지, 분리
- authstore.c: 등록 정보와 비휘발성 실패 카운터(Fail-Closed 구조)를 관리 재부팅을 통한 해킹 시도를 원천 차단

6단계 브링업

1단계 · 보드

보드만 연결해 기본 동작을 확인합니다.

2단계 · 버튼

버튼 입력을 테스트, UART 헤더 확인

3단계 · OLED

디스플레이 출력 확인

4단계 · 피에조

센서 임계값 조절
사용자 등록 & 인증 테스트

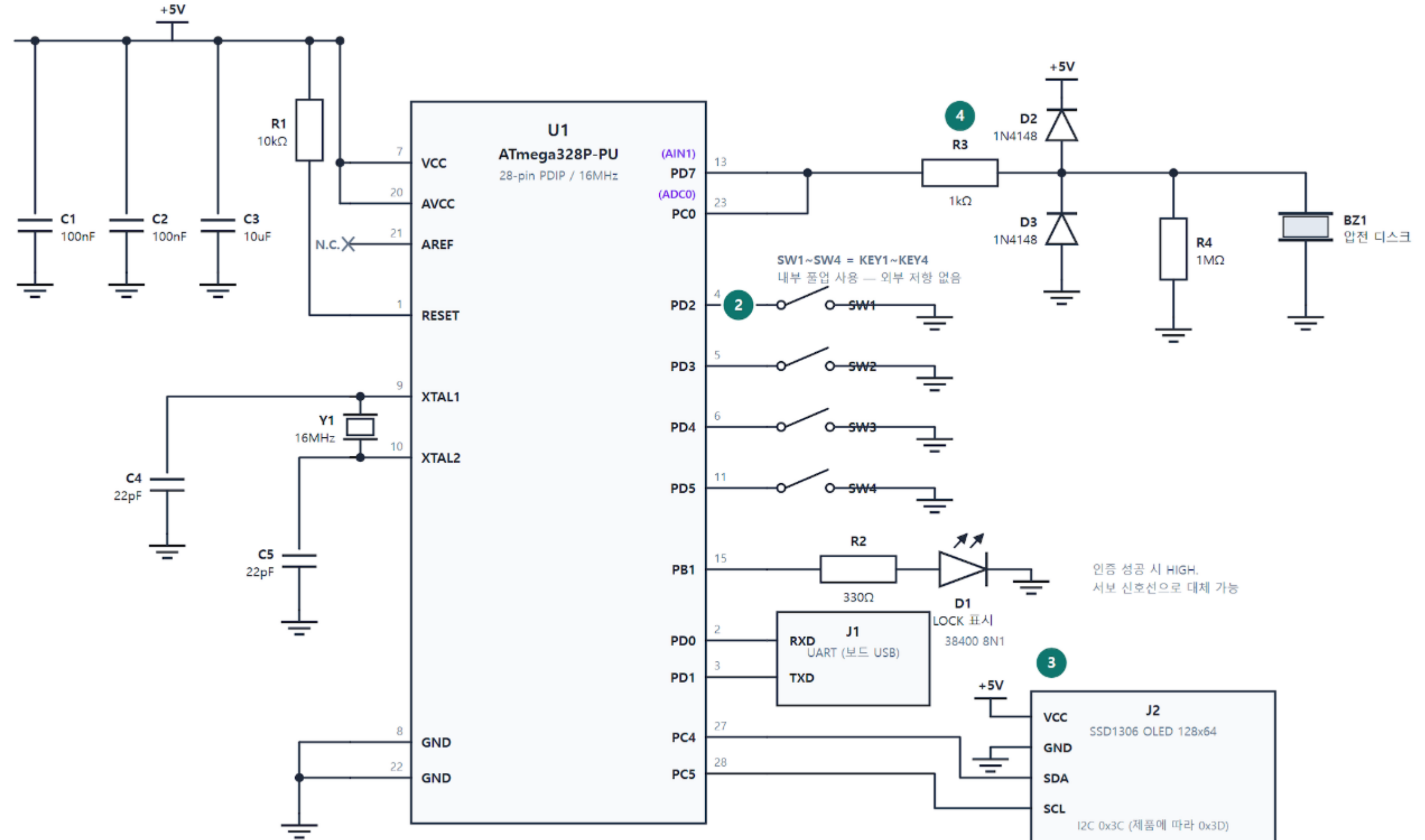
5단계

사용자 등록 확인

6단계 · 액추에이터

액추에이터 동작을 확인합니다.

세분화된 브링업 단계와 이를 기반으로 한 단계별 브레드보드 도면을 통해 오류시 직전 단계 연결 확인
→ 효율적인 오류 검증 시스템



6단계 브링업 - 1단계

1단계 · 보드

보드만 연결해 기본 동작을 확인합니다.

2단계 · 버튼

버튼 입력을 테스트, UART 헤더 확인

3단계 · OLED

디스플레이 출력 확인

4단계 · 피에조

센서 임계값 조절
사용자 등록 & 인증 테스트

5단계

사용자 등록 확인

6단계 · 액추에이터

액추에이터 동작을 확인합니다.

링잉 불응기 250ms 상향 조정

초기에는 50ms이었으나 실물 디스크가 오래 올려
200ms에서도 잔향 잡힘 → 250ms까지 브링업 기준값 상향 조정

링잉 불응기 최종 값 FRR/FAR 곡선을 통해 결정

ADC 프리스케일러 `/16`의 의도적 유지

브링업 1단계 결과 데이터시트 권장 10비트 정확도 범위(50~200 kHz) 벗어남
그러나 이 시스템에 필요한 것은 '절대적인 전압 정확도'가 아니라 '빠른 속도로 타격의 최대치를 놓치지 않는 것'
→ 프리스케일러를 `/16` (1 MHz)으로 유지하여 변환 시간을 13 μ s로 줄이고,
2ms 창 동안 약 150개의 표본을 빠르게 취득
→ 타격의 피크 감지 ↑

6단계 브링업 - 2단계

1단계 · 보드

보드만 연결해 기본 동작을 확인합니다.

2단계 · 버튼

버튼 입력을 테스트, UART 헤더 확인

3단계 · OLED

디스플레이 출력 확인

4단계 · 피에조

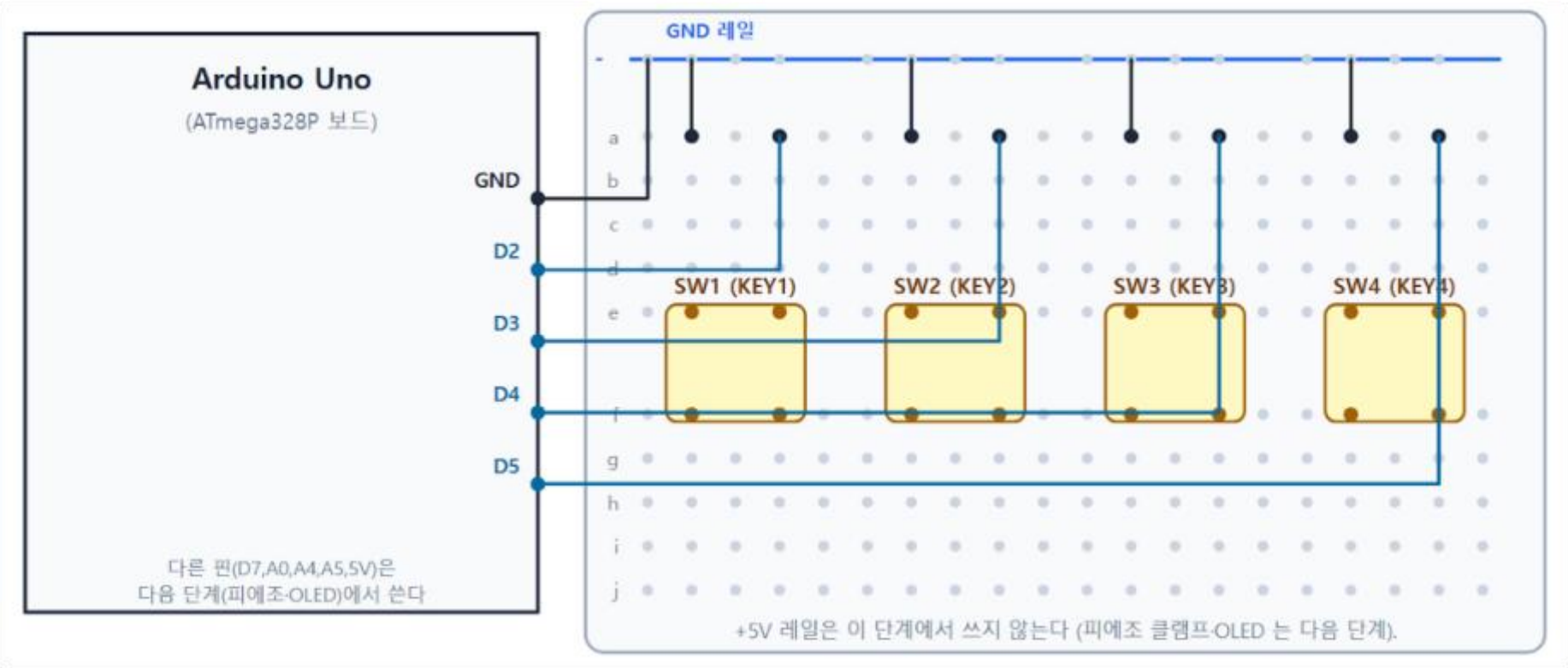
센서 임계값 조절
사용자 등록 & 인증 테스트

5단계

사용자 등록 확인

6단계 · 액추에이터

액추에이터 동작을 확인합니다.



```
sh tools/run_tests.sh — 실행 결과
=== knockfp / classify 단위 테스트 (avr-gdb AVR 시뮬레이터 실행) ===

PASS 균등 리듬 - 상태 OK / 지문 유효 / 세 성분 동일
PASS 균등 리듬 - 각 성분 약 333 / 합이 1000 근처
PASS 템포 불변 - 기준 지문 (250,500,250)
PASS 템포 불변 - 1.5배 느려도 동일
PASS 템포 불변 - 2배 빨라도 동일
PASS 배제 - 간격이 너무 짧음 / 전체가 너무 짧음 / 너무 김
PASS 경계 - 하한 바로 위는 통과
PASS 오버플로 - 최대 길이(5000ms) 입력 통과
PASS 오버플로 - 시프트 후에도 비율 보존 (200,400,400)
PASS 등록 - 중심점 평균 / 최대 흩어짐 계산
PASS 판정 - 동일 리듬은 사용자 0으로 인식 (거리 0)
PASS 판정 - 느린 템포의 같은 리듬도 통과
PASS 판정 - 다른 리듬은 거부 (2FA 2단계 방어)
PASS 판정 - 임계값 상한에서도 다른 리듬은 거부
PASS 판정 - 미등록 상태에서는 어떤 리듬도 거부
PASS 판정 - 무효 지문은 임계값과 무관하게 거부

검사 26개 중 실패 0개
```

시뮬레이터 기반 자동화 단위 테스트

하드웨어가 없어도 avr-gdb AVR 시뮬레이터를 연동하여 run_tests.sh 스크립트로 26개의 단위 테스트 자동 실행 성공 26개, 실패 0개

시스템을 통한 하드웨어 독립성

템포 정규화 모듈(knockfp.c)에 하드웨어 헤더(<avr/io.h>)가 들어가는 순간 빌드가 실패하도록 구조적으로 강제하여 구별

6단계 브링업 - 3단계 (논리적 결함)

1단계 · 보드

보드만 연결해 기본 동작을 확인합니다.

2단계 · 버튼

버튼 입력을 테스트, UART 헤더 확인

3단계 · OLED

디스플레이 출력 확인

4단계 · 피에조

센서 임계값 조절
사용자 등록 & 인증 테스트

5단계

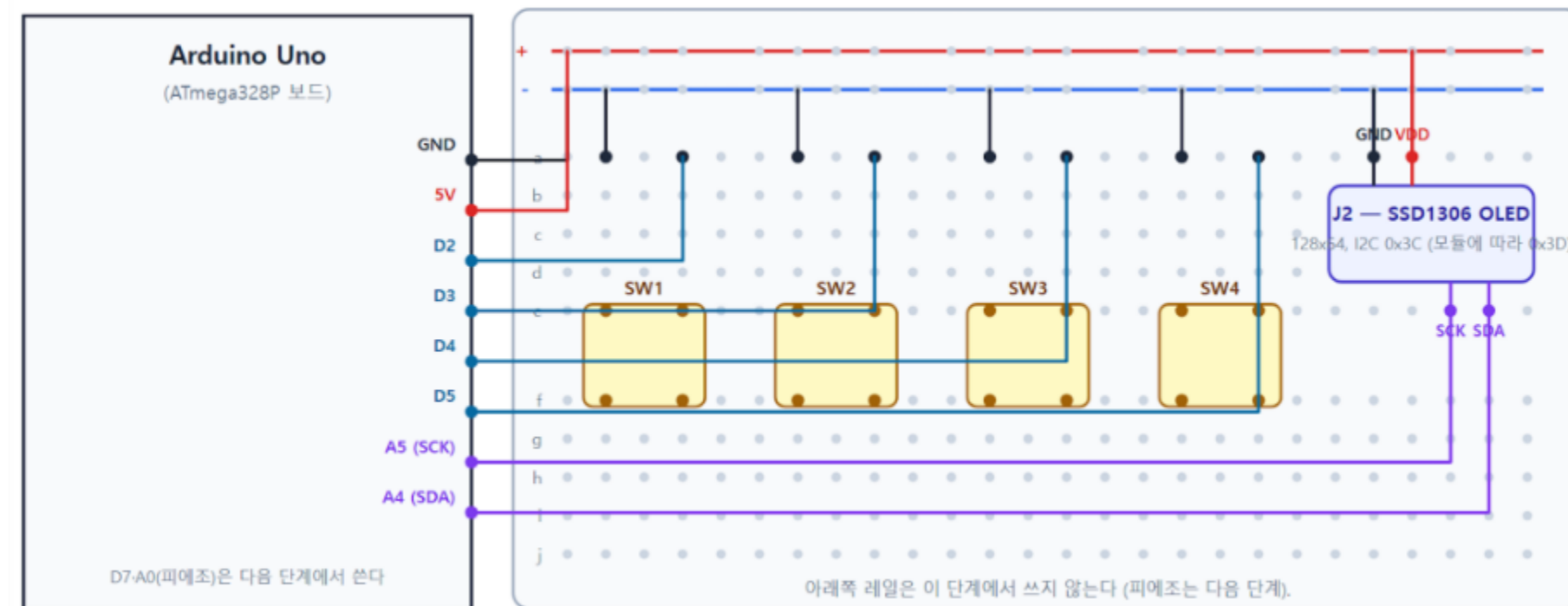
사용자 등록 확인

6단계 · 액추에이터

액추에이터 동작을 확인합니다.

단위 테스트와 코드 검토로 논리적 결함 3건 + 브링업으로 물리적 결함 3건 발견

: 시뮬레이터로는 실물의 노이즈나 하드웨어 초기화 문제를 알 수 없고, 실물로는 알고리즘의 예외 상황을 다 잡기 어렵다는 점 시사



판정 기준 수정 + 회귀 테스트 등록

다른 리듬 오인식 후 통과

→ 임계값 상한 조정(60000 → 20000)

해당 조합을 단위 테스트에 회귀 테스트 케이스로 고정

다중 사용자 임계값 격리 구조

인증 임계값 변수(`threshold\_d2`)가 공용 스칼라 하나로 선언되어 기존 사용자의 인식률(FRR/FAR)이 감소

→ 구조체를 배열(`AUTH\_USERS`) 형태로 전면 개편하여 사용자별 고유한 임계값과 중심점 독립, EEPROM 배치 최적화

인터럽트 경합에 따른 락(Lock) 방지

32비트 시각에 의존하면서 전역 인터럽트(`sei()`)가 꺼진 상태에서는 진전 불가하여 밴드갭 안정화 대기 함수가 인터럽트 상태에 의존하여 멈춰버릴 위험 존재

→ 16비트 모듈러 뺄셈 방식으로 코드를 변경하여 인터럽트 상태와 무관하게 안전하게 동작하도록 개선

6단계 브링업 - 3단계 (물리적 결함)

1단계 · 보드

보드만 연결해 기본 동작을 확인합니다.

2단계 · 버튼

버튼 입력을 테스트, UART 헤더 확인

3단계 · OLED

디스플레이 출력 확인

4단계 · 피에조

센서 임계값 조절
사용자 등록 & 인증 테스트

5단계

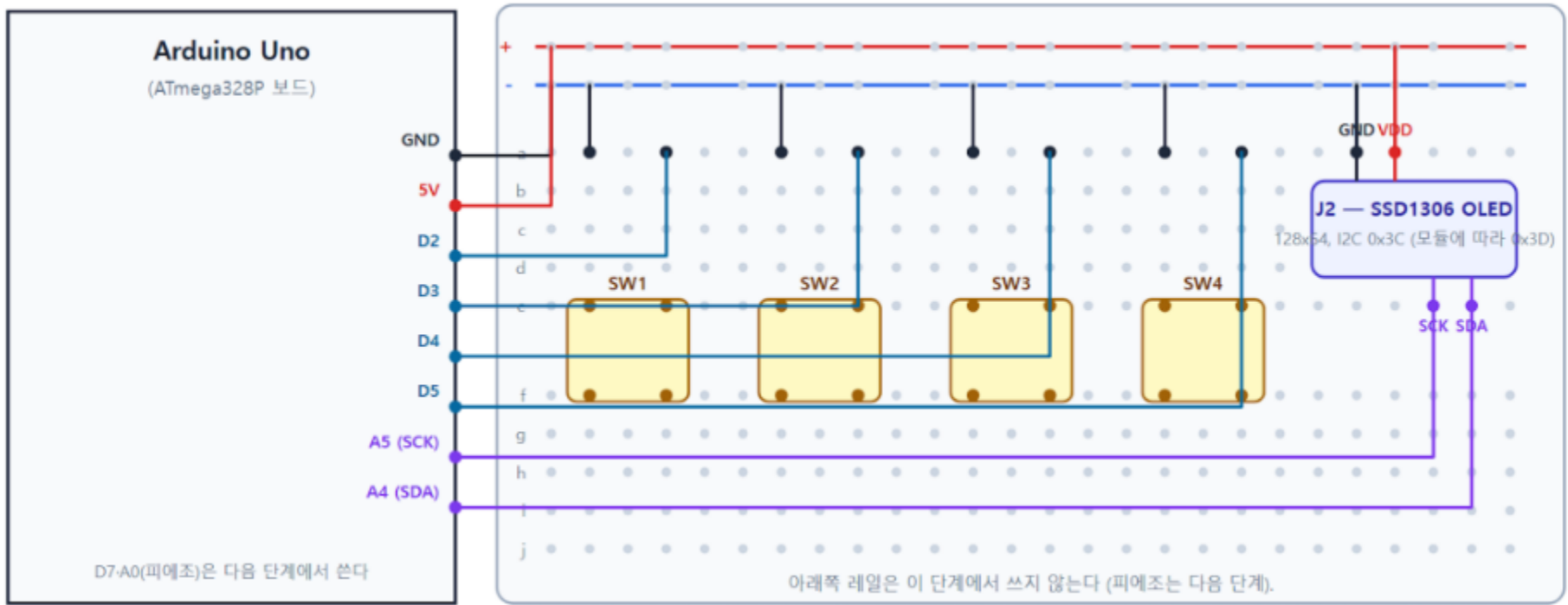
사용자 등록 확인

6단계 · 액추에이터

액추에이터 동작을 확인합니다.

단위 테스트와 코드 검토로 논리적 결함 3건 + 브링업으로 물리적 결함 3건 발견

: 시뮬레이터로는 실물의 노이즈나 하드웨어 초기화 문제를 알 수 없고, 실물로는 알고리즘의 예외 상황을 다 잡기 어렵다는 점 시사



등록 모드 탈출 경로 생성

비정상적인 입력 반복 시 탈출 가능 예외 처리 누락
→ 연속으로 5회 실패할 경우 강제로 등록을 취소하고 대기 화면으로 탈출할 수 있는 안전 경로 구축

새 칩 초기화 시 오작동 방지 로직 추가

EEPROM을 시스템이 '카운터 손상'으로 오판하여 공장 출하 상태의 새 칩을 켜면 락아웃 부터 실행
→ 출하상태의 오류인 `0xFF / 0xFF` 조합을 정상적인 상태로 별도로 분기

첫 노크 무시 제거

링잉 불응기 때문에 캡처 시작 직후 첫 번째 노크가 불응기에 걸려 무시되는 사각지대 발생
→ 시작 시각을 불응기 이전 시점으로 보정하여 첫 노크가 누락되지 않도록 수정

6단계 브링업 - 4&6 단계

1단계 · 보드

보드만 연결해 기본 동작을 확인합니다.

2단계 · 버튼

버튼 입력을 테스트, UART 헤더 확인

3단계 · OLED

디스플레이 출력 확인

4단계 · 피에조

센서 임계값 조절
사용자 등록 & 인증 테스트

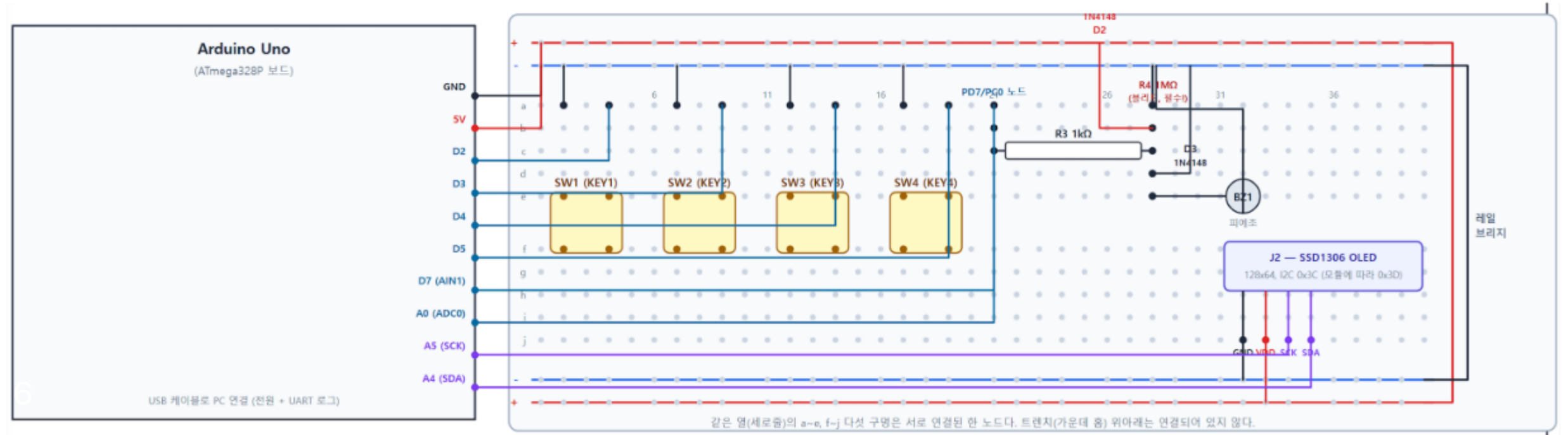
5단계

사용자 등록 확인

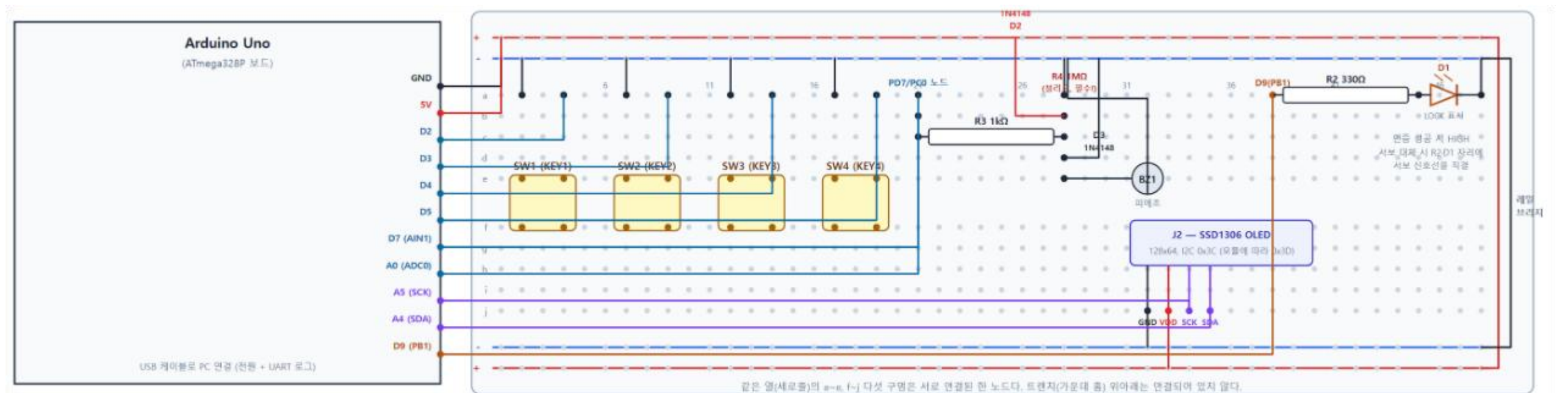
6단계 · 액추에이터

액추에이터 동작을 확인합니다.

4단계



6단계



단위 상세 - 1. 피에조 입력부

하드웨어 구성 및 특징

동시 연결: 비교기, 타이머, ADC 세 주변장치를 동일한
 피에조 센서 신호원에 동시 연동
 센서 적용: 엄선 디스크(단단한 면 고성 + 댐핑새 구조)를
 사용하여 타격 신호 감지력 향상

핵심 로직: 진폭 게이트와 ADC의 역할

한계 극복: 비교기는 Threshold V 넘음 여부만 판단 가능하므로
 실제 타격의 세기 측정용 ADC 병행 필수
 샘플링 및 필터링: 캡처 직후 약 150분을 샘플링하여 최댓값 도출
 오인식 방지: AMP_MIN < 70 미만인 경우 생활 진동으로 간주
 노크로 인정하지 않음 (의도적 타격과 구분)

주요 펌웨어 설정 포인트

디지털 입력 버퍼 차단 및 1.1V 밴드갭 레퍼런스 활용으로
 전원·온도 변동에 따른 노이즈 제어

```
knock.c — 주변장치 초기화 (주석 요약)

/* 디지털 입력 버퍼 차단 - 아날로그 전압이 버퍼를 흔드는 것을 막는다 */
DIDR1 |= (1 << AIN1D);
DIDR0 |= (1 << ADC0D);

/* ACME=0 - AIN1(PD7)을 비교기 음극 입력으로 고정.
   ACME=1 방식은 ADEN=0(=ADC 끄)이어야 해서 쓸 수 없다. */
ADCSRB &= ~(1 << ACME);

/* ACBG=1 : 내부 1.1V 밴드갭을 양극 입력으로 - 전원·온도에 흔들리는
   논리 문턱값 대신 쓴다. 가변저항 불필요.
   ACIC=1 : 비교기 출력 → Timer1 Input Capture. 0이면 캡처가 아예 안 난다. */
ACSR  = (1 << ACBG) | (1 << ACIC);

/* REFS0=1 AVCC / ADLAR=1 상위 8비트만 읽어 원자적 읽기 부담 제거
   ADPS2 → /16 = 1MHz : 변환 1회 13us, 2ms 창에서 약 150표본 */
ADMUX = (1 << REFS0) | (1 << ADLAR) | PIEZO_ADC_CH;
ADCSRA = (1 << ADEN) | (1 << ADPS2);

/* ICNC1=1 노이즈 캔슬러(4표본 일치) / ICES1=0 하강 에지
   평상시 V(PD7)≈0V < 1.1V 이므로 ACO=1, 노크로 넘으면 1→0
   CS11 → /8 = 2MHz → 분해능 500ns, 오버플로 주기 32.768ms */
TCCR1A = 0;
TCCR1B = (1 << ICNC1) | (1 << CS11);
TIMSK1 = (1 << ICIE1) | (1 << TOIE1);

/* 밴드갭 안정화 10ms 대기 - sei() 이전이라 32비트 시각을 쓰지 않고
   16비트 모듈러 뺄셈으로 센다 (결함 @ 수정) */
{ uint16_t t0 = TCNT1;
  while ((uint16_t)(TCNT1 - t0) < 20000) ; }
```

단위 상세 - 2. 버튼 입력부

하드웨어 구성 및 특징

내부 풀업 적용: 탭트 스위치 4개를 MCU 내부 풀업으로 처리
설계 장점: 외부 저항을 완전히 제거하여 부품 수 및 배선 간소화

핵심 설계 및 알고리즘 포인트

디바운스: 커패시터 없는 시간 기반 디바운스(20ms 주기 폴링)로
1틱 미만의 채터링 변화 제거

LONG 이벤트 시점: 펄 때가 아닌 누르면(2s) 등록 화면으로 전환
중복 이벤트 방지: LONG이 발생한 키는 릴리즈 SHORT 이벤트를
내지 않도록 처리해 두 번 입력되는 오류 차단

입력 규칙: 4자리 순서 입력(총 256가지 조합),
10초 이상 무입력 시 자동 상태 취소



knock.c — 주변장치 초기화 (주석 요약)

/* 디지털 입력 버퍼 차단 - 아날로그 전압이 버퍼를 흔드는 것을 막는다 */

DIDR1 |= (1 << AIN1D);

DIDR0 |= (1 << ADC0D);

/* ACME=0 - AIN1(PD7)을 비교기 음극 입력으로 고정.

ACME=1 방식은 ADEN=0(=ADC 끄)이어야 해서 쓸 수 없다. */

ADCSRB &= ~(1 << ACME);

/* ACBG=1 : 내부 1.1V 밴드갭을 양극 입력으로 - 전원·온도에 흔들리는
논리 문턱값 대신 쓴다. 가변저항 불필요.

ACIC=1 : 비교기 출력 → Timer1 Input Capture. 0이면 캡처가 아예 안 난다. */

ACSR = (1 << ACBG) | (1 << ACIC);

/* REFS0=1 AVCC / ADLAR=1 상위 8비트만 읽어 원자적 읽기 부담 제거

ADPS2 → /16 = 1MHz : 변환 1회 13us, 2ms 창에서 약 150표본 */

ADMUX = (1 << REFS0) | (1 << ADLAR) | PIEZO_ADC_CH;

ADCSRA = (1 << ADEN) | (1 << ADPS2);

/* ICNC1=1 노이즈 캔슬러(4표본 일치) / ICES1=0 하강 에지

정상시 V(PD7)≈0V < 1.1V 이므로 ACO=1, 노크로 넘으면 1→0

CS11 → /8 = 2MHz → 분해능 500ns, 오버플로 주기 32.768ms */

TCCR1A = 0;

TCCR1B = (1 << ICNC1) | (1 << CS11);

TIMSK1 = (1 << ICIE1) | (1 << TOIE1);

/* 밴드갭 안정화 10ms 대기 - sei() 이전이라 32비트 시각을 쓰지 않고

16비트 모듈러 펄셈으로 센다 (결함 @ 수정) */

{ uint16_t t0 = TCNT1;

while ((uint16_t)(TCNT1 - t0) < 20000) ; }

단위 상세 - 3. OLED/ UART 출력부

라이브러리 없는 고효율 OLED 제어

라이브러리 없는 고효율 OLED 제어:

TWI 마스터를 레지스터로 직접 구현하여 Flash 메모리 사용량 최소화

UART 38400 8N1 실시간 로그 송신

UART 38400 8N1 실시간 로그 송신:

인증 시도마다 지문 성분, 원본 간격, 진폭, 제곱거리, 임계값, 판정 결과 (result)를 CSV 포맷으로 보드의 USB 단자로 출력

사용자와 실험자 경로의 물리적 분리

사용자와 실험자 경로의 물리적 분리:

화면(OLED) 출력이 중단되거나 에러가 나도 UART 로그 데이터 수집은 백그라운드에서 계속 유지되도록 설계

```
seq,mode,pin_user,knock_user,d2,thr,n0,n1,n2,
g0ms,g1ms,g2ms,a0,a1,a2,a3,result
5,AUTH,0,0,1204,6800,248,503,249,
205,404,199,140,155,141,149,GRANT
6,AUTH,0,,38210,6800,331,334,335,
266,268,271,132,140,135,144,DENY
```



단위 상세 - 4. 상태 LED

라이브러리 없는 고효율 OLED 제어

MCU 출력 핀(PB1), 전류 제한 저항(R2, 330Ω , 약 9mA),
상태 표시 LED(D1, 5mm)

서보 모터 구동을 후순위로 미룬 이유

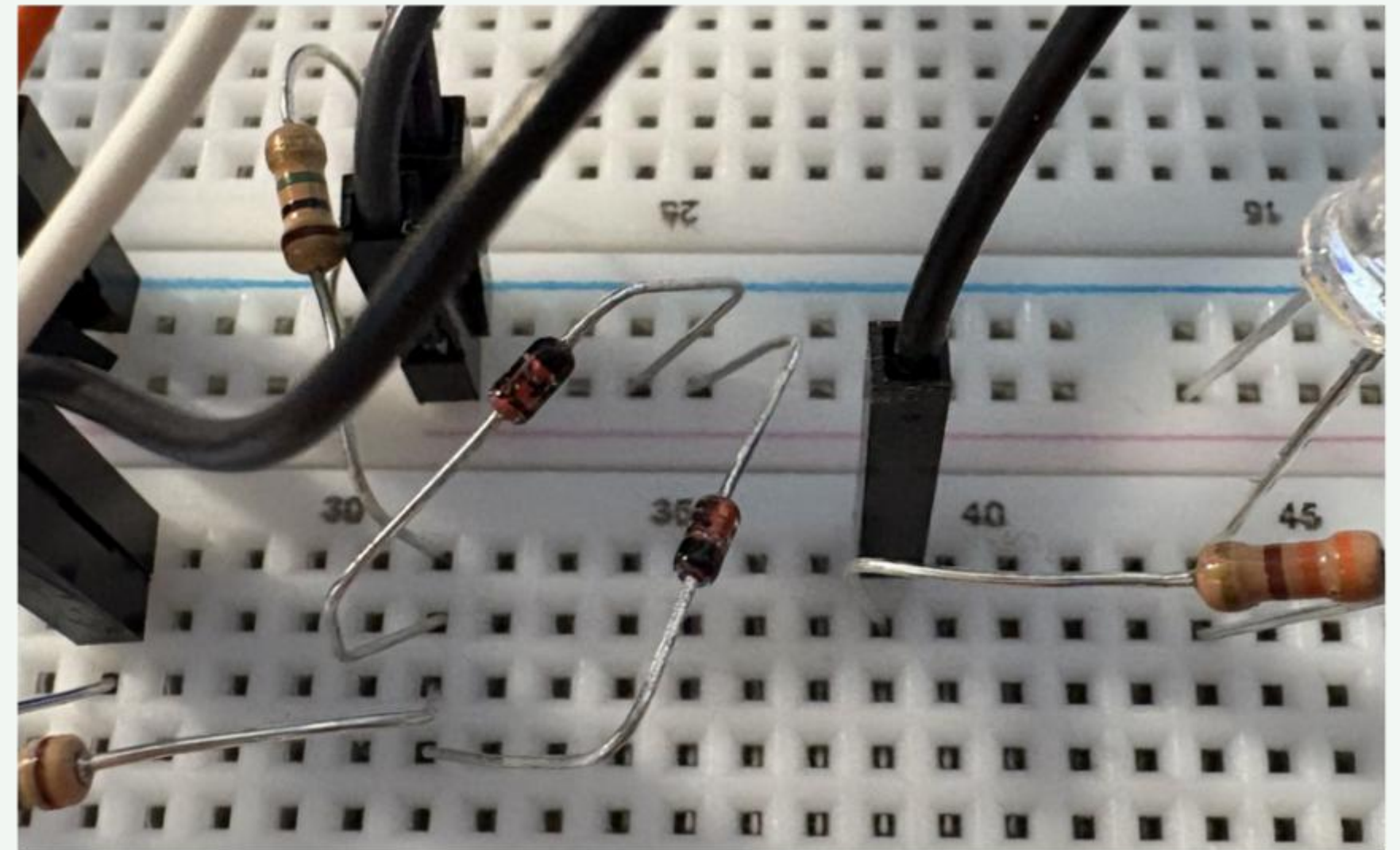
서보 모터는 순간적으로 수백 mA를 소모, 5V 레귤레이터 전압을
떨어뜨려 MCU 리셋을 유발
브링업 단계에서는 전기적으로 안전한 LED 신호 경로를 우선 확정

3초간의 안전한 해제 상태 유지

인증 성공 시 PB1 HIGH 및 3초 자동 잠금 복귀를 적용
사용자가 문을 열고 들어가는 동안 타인이 곧바로 재등록 슬롯에 접근하는
보안 사고 방지

시연 속도 및 실험 효율 기여

시상태 LED를 통해 인증 결과를 눈으로 즉시 확인할 수 있어 타인 모사 실험
등의 진행 속도 대폭 향상



단위 상세 - 5. 등록경로 -임계값 유도

데이터 기반 동적 임계값 유도

사람이 직접 값을 지정하지 않고, 5회 반복 입력된 표본의 중심점과 중심점에서 가장 멀리 벗어난 최대 제공거리를 기반으로 임계값 산출

안전 마진 적용

평균 거리가 아닌 최악의 변동성까지 수용하되, 상한선을 넘으면 타인 통과를 막기 위해 등록을 과감히 거부

성분당 변동 허용 범위 최적화

임계값 상한 20,000, 허용 반경 141 기준으로 성분당 약 8% 수준의 변동 허용 (실제 동일 리듬 재현 시 성분당 변동은 3~5% 수준)

사용자별 독립된 임계값 슬롯 구조

공용 스칼라 변수가 아닌 배열(g_db.threshold_d2[slot])로 개편하여, 새로운 사용자를 추가 등록해도 기존 사용자의 인식률(FRR/FAR)이 흔들리지 않도록 격리

```
main.c — enroll_commit()

static uint8_t enroll_commit(void) {
    uint16_t centroid[CLASSIFY_CHANNELS];
    uint32_t spread, th;

    /* 5회 반복 입력의 평균 = 이 사용자의 중심점 */
    knockfp_centroid(g_enroll_fp, ENROLL_REPEAT, centroid);

    /* 표본들이 중심점에서 벗어난 '최대' 제공거리 */
    spread = knockfp_spread_d2(g_enroll_fp, ENROLL_REPEAT, centroid);
    th = spread * THRESHOLD_MARGIN; /* x2 */

    /* 흩어짐이 너무 크면 그 사람의 리듬은 재현성이 없다는 뜻이다.
       임계값을 무리하게 넓혀 저장하면 타인도 통과하게 되므로
       등록 자체를 거부한다. */
    if (th > AUTH_THRESHOLD_MAX) return 0; /* 20,000 */
    if (th < AUTH_THRESHOLD_MIN) th = AUTH_THRESHOLD_MIN;

    ...

    /* 이 사용자 슬롯만의 임계값.
       이미 등록된 다른 사용자에게는 영향이 없다. (결합 @ 수정) */
    g_db.threshold_d2[g_enroll_slot] = th;

    authstore_save(&g_db);
    authstore_fail_set(0);
    return 1;
}
```

타인 모사 시도 실험 결과

등록자와 제3자의 거부율 및 평균 리듬 오차 비교

시도자	순서 인지 여부	시도 횟수	거부 횟수	거부율	평균 리듬 오차 (CV)	비고
본인 (등록자)	-	10	1	10%	0.08	정상 사용자, 낮은 CV로 대부분 통과
제3자 A	O (순서 전달받음)	10	9	90%	0.31	순서는 맞췄지만 타이밍 간격이 큼
제3자 B	O	10	10	100%	0.42	리듬 재현 실패, 전부 거부
제3자 C	O	10	8	80%	0.27	2회 우연히 임계값 안쪽으로 통과

— O: 순서 인지

계획하였으나 해결하지 못한 부분

1

압전 저항 억제

문제 및 원인

50~400ms 실측 결과 부저용 디스크 특성상 잔향이 지속되어 250ms 불응기로도 완전히 제거되지 않음

해결 계획

댐핑재 재질·두께 변경, 노크 감지 전용 디스크 교체, 남은 잔향의 임계값 흡수 처리

2

FRR & FAR

문제 및 원인

잔향이 남은 상태의 데이터는 오염되어 무의미하므로, 핵심 실증 실험인 FRR/FAR 측정을 의도적으로 보류함

해결 계획

항목 1(잔향 해결) 완료 후 동일 사용자 10회 X 3세션 수집(FRR) 및 제3자 3인 X 20회 시도(FAR) 진행

3

임계값 상한값 확정

문제 및 원인

현재 값은 정규화 스케일에서 역산한 합리적 초기값일 뿐, 실측 곡선으로 최종 검증된 값이 아님

해결 계획

FRR/FAR 교차 영역 도출 후 상한 재설정 및 회귀 테스트 갱신

4

ADC 검증

문제 및 원인

ADC 프리스케일러(/16)의 진폭 안정성을 정량화하지 못 함

해결 계획

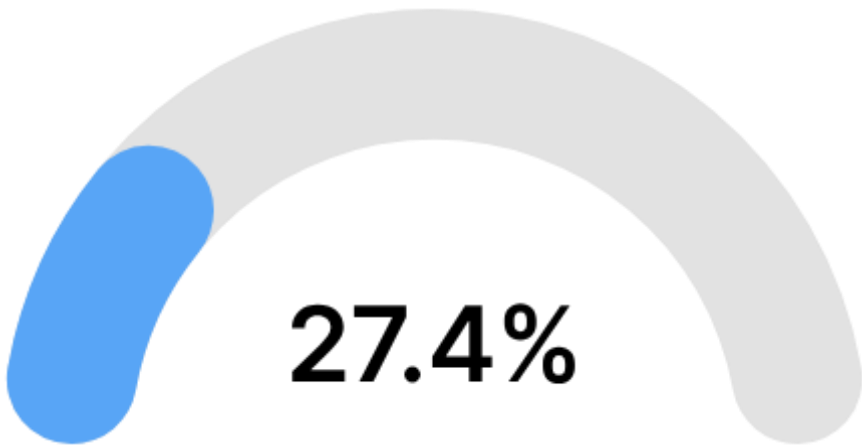
동일 세기 타격 30회의 진폭 분산 비교 측정 및 별도 전원 분리

리소스 사용 현황

1

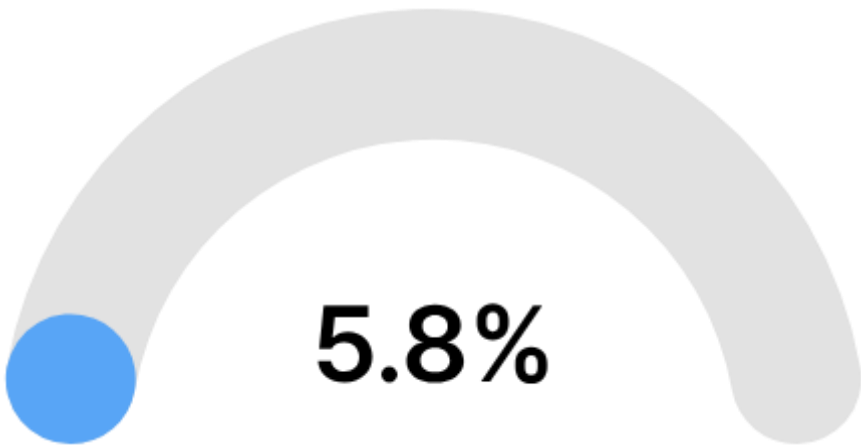
프로그램 메모리

32KB 플래시 대비 사용률을 Microchip Studio 빌드 결과 2KB SRAM 대비 데이터 메모리 사용률을
의 .text, .data, .bss로 확인



2

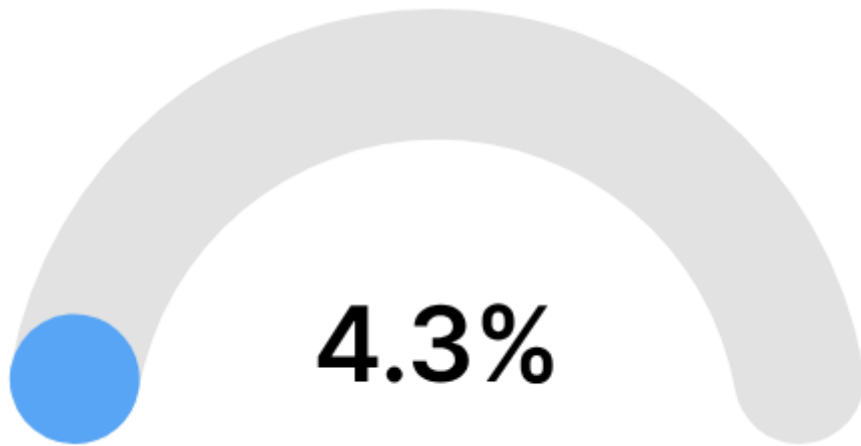
데이터 메모리



3

비휘발성 메모리

EEPROM에 등록된 정보와 실패 카운터



```
=== 메모리 사용량 (ATmega328P: Flash 32768 B / SRAM 2048 B) ===
.text      8972 B   .data      0 B   .bss      118 B
Flash      8972 / 32768 B ( 27.4 %)
SRAM       118 / 2048 B (  5.8 %)
```

```
$ avr-size -A build/knock2fa.elf
build/knock2fa.elf :
section            size      addr
.data              0      8388864
.text             8972         0
.bss               118     8388864
```

부품가격표

시제품 제작을 위한 부품 구성 및 비용

단위: 원

품목	사양	수량	단가	금액
Arduino Uno 호환보드	ATmega328P	1	6,000	6,000
택트 스위치	6×6 mm 4핀	4	100	400
I²C OLED	0.96" · SSD1306	1	3,500	3,500
압전(피에조) 디스크	27 mm	1	500	500
저항 1 MΩ (블리드)	1/4 W	1	20	20
저항 1 kΩ (직렬 보호)	1/4 W	1	20	20
다이오드 1N4148 (클램프)	-	2	30	60
저항 330 Ω	1/4 W	1	20	20
LED	5 mm	1	50	50
브레드보드	830핀	1	2,500	2,500
점퍼선 세트	M/M · M/F	1식	2,000	2,000
시제품 총 부품 원가				15,070 원

감사합니다.

